Physical AI 応用編 2026　最終課題 提出レポート

KumaWatch Patrol：AI による熊出没リスク巡回ロボット

Risk-Aware Robotic Patrol for Bear Encounters

提出者	**Todalaba**	**提出日**	2026 年 11 月
動画 URL	https://www.youtube.com/watch?v=k5Kr4eU_tf8	**コード**	github.com/Global-jogasaki/kumawatch-patrol

1　背景と目的

近年、熊の人里への出没が増え、自治体は通学路やごみ集積所などの見回りに追われている。しかし人員には限りがあり、「明日はどこを見に行くべきか」を限られた数だけ選ぶ必要がある。筆者らの研究 KumaWatch [7] では、山形県・秋田県の公開出没データを 10 km グリッドに集計し、翌日の出没リスクが高いセルを上位 K 個選ぶ問題として複数の予測手法を比較してきた。一方、予測はあくまで地図上の数値であり、実際に現場を確認し、熊がいれば安全に行動する部分は人に委ねられている。本作品では、この「予測から現場確認まで」をロボットで具体化する。

1. AI の出没リスク予測に基づいて、巡回可能な数（$K = 3$）の地点と巡回順を自動で決める。
2. Navigation2 で候補地点を自律巡回し、カメラ画像を AI で判定して熊を見つける。
3. 熊を見つけたら追跡せずに停止し、位置・時刻・信頼度を記録してから、熊から離れた安全地点へ退避する（安全側の行動）。
4. 固定ルートの巡回と同じ条件で比較し、AI で地点を選ぶ効果を示す。

2　提案システム

提案システム KumaWatch Patrol は、ROS 2 Humble [4]・Navigation2 [5]・Gazebo Classic 11 [6] 上の移動ロボット（TurtleBot3 / Happy Mini）で、次の流れを自動で実行する（図 1）。① AI（TTM）が 5 候補地点の熊出没リスクを予測する。② リスク・重要度・移動距離から巡回する 3 地点と順番を決める。③ Nav2 で自律巡回する。④ AI（YOLOv8）がカメラ画像から熊を検知する。⑤ 停止して記録し、熊から離れた安全地点へ退避してから巡回を続ける。⑥ 詰所に戻り、結果を端末・RViz・ファイルに出力する。AI を使うのは①と④で、②の地点選択は AI の予測を入力とする規則（効用最大化）である。町と地点は架空で、熊・シカ・建物は基本図形で作った。リスクは公開データに基づく TTM の予測値、熊判定の学習画像はシミュレータで作った合成画像である。

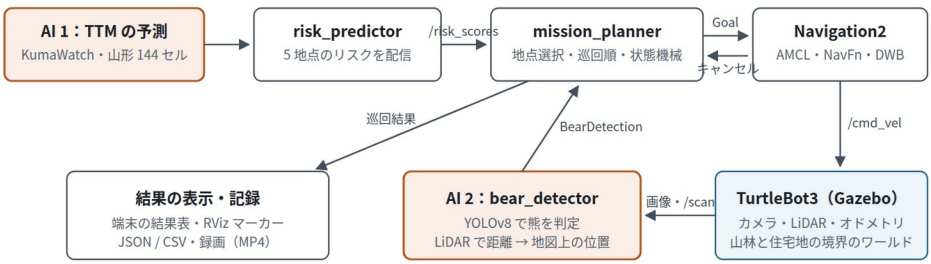


図 1　システム構成。　橙の枠が AI を使う部分。ノード間は ROS 2 のトピック（リスク・熊検知は独自メッセージ）と Nav2 の NavigateToPose アクションでつながる。

リスクだけでなく、社会的重要度と移動コストを考えた効用で地点を選ぶ。

$$U_i = w_i\, p_i - \lambda\, d_i$$

p_i はリスク、w_i は重要度（通学路 1.2、住宅地 1.1、他 1.0）、d_i は詰所からの距離 [m]、$\lambda = 0.02$ である。U_i の上位 3 地点を選び、詰所から回って戻る総移動距離が最小になる順を全順列で求める。今回は P2（林縁）→ P4（ごみ集積所）→ P3（通学路）となった。比較用に、リスクだけで選ぶ方式と固定ルートも実装した。

`mission_planner` は Nav2 の NavigateToPose アクションで各地点へ移動し、到着後に一定時間周囲を確認する。熊の検知を受け取ると、ゴールをキャンセルしてゼロ速度で停止し、熊の位置・時刻・距離・信頼度を記録する。熊を追跡せず、熊と反対の方向へ 3 m 離れた地点を安全地点として計算し、Nav2 で退避してから巡回を再開する。退避先への経路が得られない場合、本試作では退避を打ち切って巡回を継続する。これは安全制御上の課題である（5・6・8 章）。

3　AI の利用方法

3.1　AI 1：熊出没リスクの予測（KumaWatch の TTM）

リスク予測には、KumaWatch が公開している **IBM Granite TinyTimeMixer [1]（TTM 1536-96-R2）**の予測を使った。TTM は大量の時系列で事前学習された時系列予測の基盤モデルで、KumaWatch では山形県 144 セルの日別出没件数の履歴から、学習に使っていない 2025 年の各日の出没をゼロショットで予測している。作品の町は架空だが、5 地点のリスクには実在セルの予測値を割り当てた（表 2）。割り当ては次の規則による：2025 年 6 月 7 日に実際に出没があった 2 セルを P2・P4 とし、Gazebo でもそこに熊を置く。出没がなかったセルからは、TTM 順位が最も高いもの（P3、AI の空振り）、40 位付近（P5）、100 位付近（P1）を選んだ。リスクは県内 144 セル中の順位を百分位にした値である。

地点	環境	セル	TTM 順位	リスク	実際の出没	Gazebo の熊
P4	ごみ集積所	5_1	1 / 144	1.000	あり	あり
P3	通学路	4_1	2 / 144	0.993	なし（空振り）	なし
P2	林縁	1_10	4 / 144	0.979	あり	あり
P5	河川沿い	6_0	40 / 144	0.727	なし	なし（シカ）
P1	住宅地	1_0	93 / 144	0.357	なし	なし

表 2　5 地点に割り当てた TTM の予測（2025 年 6 月 7 日、山形県 10 km グリッド）。

3.2　AI 2：カメラ画像からの熊判定（YOLOv8）

COCO で学習済みの YOLOv8 [2] にも「bear」クラスはあるが、Gazebo の熊（基本図形の組み合わせ）は信号機や人と誤認された（図 2 左）。そこで Gazebo の画像で**追加学習（ファインチューニング）**した。カメラと熊・シカをランダムな位置に動かしながら 700 枚を撮影し、熊の 3D 位置を画像へ投影して正解の枠を自動で付けた。木や建物で隠れる視点は除き、熊が写らない画像も 3 割含めた。学習済みの YOLOv8n を初期値に追加学習した（図 2 右）。

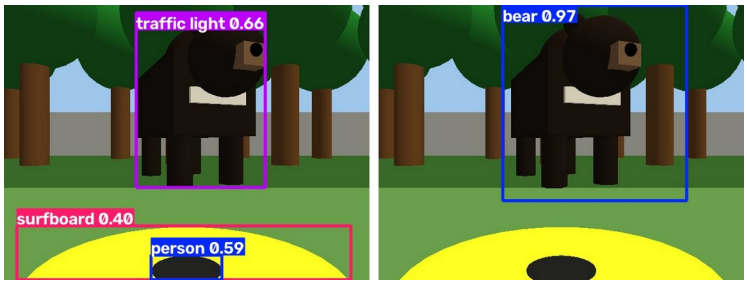


図 2　追加学習の前後。　左：COCO 学習済みの YOLOv8s は Gazebo の熊を「traffic light」と誤認する。右：追加学習後の YOLOv8n は「bear」（信頼度 0.97）と判定する。

`bear_detector` は、検出枠の横位置からカメラに対する熊の方位を、その方位の LiDAR の値から距離を求め、ロボットの自己位置と合わせて熊の地図上の位置を推定する。LiDAR が手前の木を拾った場合は、枠の高さからの推

定に切り替える。誤検出で止まらないよう、3 フレーム連続で検出し、かつ 4 m 以内に近づいたときに停止を指示する。

Precision（検証 105 枚）	Recall（検証 105 枚）	mAP@0.5	巡回中の視点での判定
0.94	0.99	0.985	11 / 11

表 3　YOLOv8 の熊判定の精度（Gazebo 画像の検証データ 105 枚と、巡回中の 11 視点）。

巡回中の 11 視点（P2・P4 への接近と到着、熊のいない P1・P3・P5、シカ、林縁など）では、熊が写る視点はすべて検出し（信頼度 0.88〜0.98）、シカ・木・建物を熊と誤認しなかった。ROS 2 ノードとしても、Gazebo のカメラ映像から熊を検出し、停止指示のメッセージを出すことを確認した。

4　ROS 2 ノード構成

ソフトウェアは 5 つの ROS 2 パッケージで構成した（表 1）。ワールドと Nav2 用の地図は、同じ定義ファイル（`world_layout.yaml`）から自動生成するので、SLAM で地図を作らなくても座標が一致する。熊とシカは地図に載せず、ロボットにとって未知の障害物として LiDAR とカメラで見つける。

ノード	役割
risk_predictor	TTM の予測を 5 地点のリスクとして配信し、RViz に棒グラフで表示
mission_planner	地点選択・巡回順の決定、Nav2 への移動指示、熊検知時の停止・記録・安全地点への退避、結果表示（状態機械）
bear_detector	カメラ画像を YOLOv8 で判定し、LiDAR の距離から熊の地図上の位置を推定
image_recorder	カメラ映像と検出枠入りの映像を状態表示付きで録画、熊検知時の静止画を保存
bear_proximity_monitor	熊の既知座標との距離で検知する簡易版（比較・動作確認用）

表 1　主なノード。　状態機械は RISK_ASSESSMENT → ROUTE_PLANNING → PATROL ⇄ OBSERVE →（EMERGENCY_STOP）→ RETURN_TO_BASE → FINISHED と遷移する。

5　Gazebo での実行結果

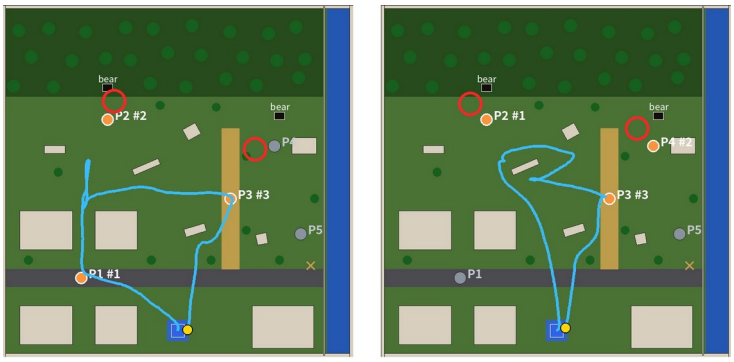
講義の演習環境（Docker イメージ airobotbook/ros2-desktop-ai-robot-book-humble、Happy Mini、GPU なし）で、Gazebo・Navigation2・YOLOv8 をすべて動かして AI ルートを 1 回走らせた（図 3、提出動画）。ロボットは詰所から P2 へ向かい、YOLOv8 がカメラ画像から熊を検知した（信頼度 0.75、距離 4.0 m）。停止して位置・時刻・信頼度を記録し、安全地点への退避を試みた。次に P4 へ向かう途中で 2 頭目を検知し（信頼度 0.82、距離 3.9 m）、停止・記録のあと安全地点へ退避した。その後 P3 に到着して熊がいないことを確認し、詰所に帰還した。3 地点すべてを確認し、熊は 2 頭、移動距離は 31.4 m、所要時間は 173 秒（シミュレーション時間）だった。なお Happy Mini のカメラ画像には車体の一部（黄色い帯とロゴの裏側）が映り込んでいたが、熊は検知できた。



図 3　Gazebo での実行の様子（提出動画より）。　左：ロボットのカメラ映像と YOLOv8 の検出枠（P2 の熊。停止距離 4 m に入る前の 5.0 m で信頼度 0.87）、右：走行記録から描いた地図。上の帯は巡回の状態（Mission 1/3・AI リスク）。

P2 では、安全地点への経路生成が障害物配置のため失敗した。退避先 (−2.7, −2.2) が建物のそばにあり、Nav2 が経路を作れなかった。本試作では巡回継続に遷移したが、実運用では安全停止、代替退避地点の再探索、管理者への通知を優先する必要がある。これは今後修正すべき安全制御上の課題である。

6　固定ルートとの比較



固定巡回 P1 → P2 → P3
熊 2 頭（うち 1 頭は P3 から偶然視認）・33.6 m

AI 巡回 P2 → P4 → P3
熊 2 頭（P2・P4 の確認中）・31.4 m

図 4　走行軌跡の比較。　講義の Docker 環境（Gazebo・Navigation2・YOLOv8）での実走行の記録から描いた。左：固定巡回（P1→P2→P3）、右：AI 巡回（P2→P4→P3）。青線は走行軌跡、赤丸は熊の発見位置、下の距離は実行ログの値。

指標	固定巡回（P1→P2→P3）	AI 巡回（P2→P4→P3）
巡回地点の AI リスク合計	2.33	2.97
確認できた地点	3 / 3	3 / 3
熊の発見数	2（うち 1 頭は P3 から P4 付近の熊を偶然視認）	2（P2・P4 の確認中）
YOLOv8 の信頼度（検知時）	0.73・0.86	0.75・0.82
安全地点への退避	2 回とも成功	P4 は成功、P2 は経路がなく失敗
移動距離（実行ログ）	33.6 m	31.4 m
所要時間（シミュレーション時間）	166.6 s	173.3 s

表 4　同じ 5 候補地点・同じ熊の配置での比較（各 1 回、詰所への帰還を含む）。講義の Docker 環境で、Gazebo・Navigation2 と、YOLOv8 によるカメラ映像での熊の検知を使った実走行の値。移動距離と所要時間は実行ログ（patrol_summary.json）の値で、動画の比較画面も同じ値である。

- **比較は各条件 1 回のシミュレーションである。**一般的な優位性を証明するものではなく、Physical AI の一連の動作（予測 → 地点選択 → 自律巡回 → 検知 → 停止・退避 → 帰還）を示すデモである。今後は熊の配置と初期条件を変えて複数回評価する。
- **固定ルートの 2 頭目は、P3 から P4 付近の熊を偶然検知したものである。**固定ルートは P4 を巡回対象にしていない。AI ルートは P4 を明示的な巡回対象として確認した。
- **退避経路の生成に失敗した場合、実運用では安全停止と管理者への通知を優先する。**本試作は P2 で巡回継続に遷移したが、これは今後修正すべき安全制御上の課題である（5・8 章）。

熊の発見数はどちらも 2 頭だが、その意味は異なる。AI 巡回は出没リスクが最も高い P4（ごみ集積所）を選び、P2・P4 の確認中に熊を検知した。固定巡回の 2 頭目は、P3（通学路）に着いたときに P4 付近の熊が偶然カメラの視野に入ったもので（3.2 m 先と判定、推定位置 (4.4, 1.8)）、AI 巡回の失敗を意味しない。AI 巡回が選んだ地点のリスク合計は 2.97 で、低リスクの P1 を含む固定巡回（2.33）より高い。移動距離と所要時間は同程度だった。

固定巡回の 1 回目は P2 で退避したあと経路計画が行き詰まり、帰還できなかった（熊 1 頭）。表は最後まで走行できた 2 回目の値である。距離は実行ログの値で、動画の比較画面も同じ値に統一した。なお 1 年を通した評価では、山形県で TTM は静的予測（過去の平均）に有意に劣る。

7　工夫した点

- **研究の予測をそのまま使う。**KumaWatch の TTM の実際の予測値を使い、実際に出没があったセルに熊を置いた AI の空振り（P3）もあえて含めた。
- **危険度だけでなく、重要度と移動距離も考える。**通学路の重みと移動コストを入れた効用で地点を選び、総移動距離が最小の順に回る。
- **ラベル付けを自動化した追加学習。**熊の 3D 位置を画像へ投影して正解の枠を自動で作り、手作業なしで 700 枚を用意した。
- **安全側の行動。**熊に近づかず 4 m 以内で停止・記録し、安全地点へ退避する。3 フレーム連続の検出で、誤検出による停止を防ぐ。
- **ワールドと地図を 1 つの定義から生成。**配置を変えても地図が一致する。自己位置推定のずれには、電柱やベンチを目印に加えて対処した。

8　制約と今後の課題

- **シミュレーションの範囲。**Gazebo 内の模擬環境であり、実環境での自己位置推定・通信・天候は検証していない。
- **熊判定は Gazebo の熊でのみ学習・評価した。**実物の熊の画像での精度は不明で、実運用には実画像での学習と評価が必要である。
- **予測と地点の対応は説明用。**TTM の予測は 10 km セル単位で、町の中の 1 地点と同じ意味ではない。日付もシナリオのために選んだ 1 日である。
- **退避できない場合の安全制御。**熊の近くで通常の巡回に戻る現在の遷移は、安全設計として弱い。安全停止、代替退避地点の再探索、管理者への通知、人による判断までの待機に改める。
- **今後。**上記の安全制御の改修、カメラへの車体の映り込みの修正、GLM-Logit や静的予測との巡回結果の比較、熊の配置と初期条件を変えた複数回の評価を行う。

9　使用ソフトウェア・参考文献

動作環境：Ubuntu 22.04、ROS 2 Humble、Gazebo Classic 11、Navigation2、TurtleBot3 / Happy Mini、Ultralytics YOLOv8。講義の Docker 環境で動作する。起動手順・使用データ・学習プログラムはリポジトリの README.md と training/ を参照。

外部ソフトウェアと素材：KumaWatch [7]（事前計算済みの 5 地点のスコアのみ収録。元データ：山形県の公開出没データ、CC BY 4.0）、IBM Granite TinyTimeMixer（Apache-2.0）、Ultralytics YOLOv8（AGPL-3.0。追加学習した重みも同ライセンス）、ROS 2・Navigation2・Gazebo（Apache-2.0）、TurtleBot3 シミュレーション（ROBOTIS、Apache-2.0）、Happy Mini（AI ロボット入門）。熊・シカ・建物・樹木は基本図形で自作。

参考文献

[1] V. Ekambaram, A. Jati, P. Dayama, et al., "Tiny Time Mixers (TTMs): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series," NeurIPS, 2024.

[2] G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," 2023. https://github.com/ultralytics/ultralytics

[3] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," ECCV, 2014.

[4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, 7(66), 2022.

[5] S. Macenski, F. Martín, R. White, J. Ginés Clavero, "The Marathon 2: A Navigation System," IROS, 2020.

[6] N. Koenig, A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," IROS, 2004.

[7] KumaWatch（熊出没リスク予測の研究。非公開の研究リポジトリ）.